

Software Validation in Safety Critical Systems

Turcu Ciprian-Stelian^a, Turcu Remus-Gabriel^a and Țăpoi Marius-Ștefan^a

^aUniversitatea Babeș-Bolyai din Cluj-Napoca, Strada Mihail Kogălniceanu 1, Cluj-Napoca 400347, Cluj, Romania

ARTICLE INFO

Keywords:

Systematic Literature Review (SLR)
Safety-Critical Software Systems
Validation and Verification Methods
Validation and Verification Methods
failure
fault

ABSTRACT

Software Validation in Safety-Critical Systems: This paper presents a systematic literature review (SLR) following Kitchenham et al.'s methodology to explore validation and verification methods in safety-critical software systems. It outlines a study design involving planning, conducting, and reporting phases, emphasizing the need for rigorous validation due to the potential severe consequences of system failures. The review addresses prevalent methods and tools, their effectiveness, and gaps in current research. It also discusses the application of these methods across various domains, highlighting the importance of early fault discovery and the integration of validation and verification techniques. The paper concludes with insights into the challenges and future directions for software validation and verification in safety-critical systems, including the adaptation to AI and dynamic operational environments.

1. Systematic literature review (SLR)

We use Kitchenham et al.'s suggested systematic literature review (SLR) methodology to describe the state-of-the-art of validation and verification methods in safety-critical software systems. [KC07]

2. Study design

. The SLR process involves three main phases:

1. Planning Phase: This phase involves:
 - Describing the motivation to conduct the SLR.
 - Specifying the research questions to be answered.
 - Developing the rules for conducting the SLR.
2. Conducting Phase: This phase follows the rules defined previously to collect primary studies and extract relevant information.
3. Reporting Phase: This phase covers the reporting of the entire SLR, synthesizing all collected information, from motivations to data synthesis, and addressing the research questions.

Following these guidelines, we present the research methodology used for this study, outlining the tasks involved in each phase.

2.1. Review need identification

The systematic review plays a crucial role in the synthesis of existing research on validation and verification techniques in safety-critical software systems. The importance

of this matter stems from the escalating intricacy and crucial significance of these systems, as their malfunction can lead to substantial repercussions, such as loss of human life.

2.2. Research questions definition

The review aims to answer the following questions:

1. What are the prevalent methods and tools used for validation and verification in safety-critical software systems?
2. How effective are these methods in identifying and mitigating potential failures in such systems?
3. What are the gaps in current research on software validation and verification for safety-critical systems?

2.3. Protocol definition

The review protocol established involves:

- Database and manual search for primary studies.
- Selection criteria based on relevance to safety-critical systems, methodological rigor, and the depth of focus on validation and verification.
- Data extraction focusing on methods, effectiveness, and application context.
- Synthesis to compare methods and determine prevalent strategies and gaps.

3. Conducting the SLR

3.1. Search and selection process

The search and selection process is designed to identify the studies that are relevant for addressing the research questions mentioned previously. The process comprises three consecutive steps, which are outlined as follows.

As an initial step, we establish two crucial components: the selection of databases and the identification of appropriate keywords to be used in the search for primary studies. At this stage, the search process is initiated by applying the

 ciprianstelian08@gmail.com (T. Ciprian-Stelian);
remusturcu73@gmail.com (T. Remus-Gabriel); tapoimarius@gmail.com (Marius-Ștefan)
 <https://www.linkedin.com/in/ciprian-turcu/> (T. Ciprian-Stelian);
<https://www.linkedin.com/in/remus-gabriel-turcu-557b00269/> (T. Remus-Gabriel); <https://www.linkedin.com/in/mariustapoi12/> (Marius-Ștefan)
ORCID(s):

Table 1

The databases and indexing systems used in the search process

Name	Type	URL
IEEE Xplore	Electronic database	https://ieeexplore.ieee.org/
HAL Science	Electronic database	https://hal.science/
SAE International	Electronic database	https://www.sae.org/
Springer Link	Electronic database	https://link.springer.com/
arXiv	Electronic database	https://arxiv.org/

predetermined keywords to each database. The subsequent step entails the process of merging.

The results derived from all databases are aggregated into a unified spreadsheet, and any redundant entries are removed. During the third phase of the research process, the aggregated studies undergo a filtration procedure that is guided by a pre-established set of criteria for determining which studies should be included and which should be excluded.

3.1.1. Database search

The search will be performed in various databases including IEEE Xplore, HAL Science, SAE International, Springer Link, and arXiv which are presented in Table 1. The keywords used for the search will be focused on topics related to "software validation," "verification," and "safety-critical systems."

3.1.2. Merging, duplicates and impurity removal

The results obtained from various databases will be combined, eliminating any duplicate entries, and excluding publications that are not relevant, such as editorials and prefaces.

3.1.3. Application of the selection criteria

In order to conduct a more refined selection of primary studies identified in the preceding step, a set of criteria is established. The specific details of these criteria are provided below. A paper will be included if it meets all the specified inclusion criteria, and it will be excluded if it meets any of the specified exclusion criteria.

Papers will be included if they:

- Are peer-reviewed articles or conference papers.
- Focus on validation and verification in safety-critical software systems.
- Are published in English.

Papers will be excluded if they:

- Do not directly address software validation and verification.
- Are not related to safety-critical systems.

3.2. Data extraction

The extraction process will involve retrieving data from the chosen papers, including information on the methodologies employed, the safety-critical systems that were examined, and the results obtained from the validation and verification procedures. In this review, a total of 10 papers have been chosen as primary studies.

3.3. Data synthesis

Article P1 discusses the Model-Based Verification, Validation and Testing (VV&T) of safety-critical embedded real-time systems, emphasizing the importance of VV&T in the system development lifecycle. The authors propose a model-based approach to VV&T, which includes various testing methods such as static testing, functional testing, and Hardware-In-The-Loop (HIL) testing, among others. They highlight the use of industrial case studies to demonstrate the feasibility of these methods.

The methods used in the article cover a range of VV&T techniques, including model checking, simulation, inspection, static analysis, and different types of in-the-loop testing. The authors utilize Matlab/Simulink as a tool for system modeling, simulation, and analysis, which facilitates the VV&T process. They also discuss the challenges associated with VV&T, such as real-world sensor modeling, actuator reliability, and the complexities of dynamic systems.

The dataset and results are not explicitly mentioned in the provided context, suggesting that the article may focus more on the theoretical aspects and methodologies of VV&T rather than on specific empirical data or outcomes from case studies. The article aims to improve the productivity and reduce the project cost in the development of safety-critical systems by employing the model-based VV&T approach.

The authors of P2 introduce a methodology and techniques aimed at formalizing and validating high-level requirements for safety-critical applications, particularly in sectors such as aerospace, avionics, and railways. They advocate for a three-phase approach, beginning with informal analysis, followed by formalization and formal validation. This methodology relies on a sophisticated formal language that combines first-order, temporal, and hybrid logic, supported by model checking and satisfiability modulo theory for validation. Notably, the authors emphasize the importance of maintaining traceability between the informal analysis and formalized counterparts, enabling incremental and modular validation approaches.

Id	Citation	Title, Authors	Year
P1	[KKW13]	Model-based verification and validation of safety-critical embedded real-time systems: Formation and tools by Khan, Arsalan H and Khan, Zeashan H and Weiguo, Zhang	2013
P2	[CRST10]	Formalization and validation of safety-critical requirements by Cimatti, Alessandro and Roveri, Marco and Susi, Angelo and Tonetta, Stefano	2010
P3	[Fei09]	Challenges in validating safety-critical embedded systems by Feiler, Peter H	2009
P4	[Fei10]	Model-based validation of safety-critical embedded systems by Feiler, Peter H	2010
P5	[PEB19]	A requirements modelling language to facilitate avionics software verification and certification by Paz, Andrés and El Boussaidi, Ghizlane	2019
P6	[LBA ⁺ 14]	Joint use of static and dynamic software verification techniques: a cross-domain view in safety critical system industries by Ledinot, Emmanuel and Blanquart, Jean-Paul and Astruc, Jean-Marc and Baufreton, Philippe and Boulanger, Jean-Louis and Comar, Cyrille and Delseny, Hervé and Gassino, Jean and Leeman, Michel and Quéré, Philippe and others	2014
P7	[AQH15]	Analyzing different validation and verification techniques for safety critical software systems by Ahmad, Waqas and Qamar, Usman and Hassan, Shoaib	2015
P8	[PFB ⁺ 21]	Optimization techniques and formal verification for the software design of boolean algebra based safety-critical systems by Perez, Jon and Flores, Jose Luis and Blum, Christian and Cerquides, Jesús and Abuin, Alex	2021
P9	[PCD ⁺ 23]	Formal verification of safety-critical aerospace systems by Paul, Saswata and Cruz, Elkin and Dutta, Airin and Bhaumik, Ankita and Blasch, Erik and Agha, Gul and Patterson, Stacy and Kopsaftopoulos, Fotis and Varela, Carlos	2023
P10	[SSS ⁺ 23]	XANDAR: Verification & Validation Approach for Safety-critical Systems by Sonigara, Balmukund and Sezer, Sakir and Siddiqui, Fahad and Weber, Raphael and Antonopoulos, Konstantinos and Panagiotou, Christos and Antonopoulos, Christos P and Keramidas, Georgios and Voros, Nikolaos and Yengec-Tasdemir, Sena Busra and others	2023

Table 2
The selected articles

Moreover, the article elucidates the methodology's practical application within an industrial project focused on railway requirements validation. By utilizing data from the European Train Control System (ETCS), the authors demonstrate the effectiveness of their approach in improving requirement quality and increasing confidence in their correctness. Additionally, they developed a tool that integrates requirements management, model-based design, and advanced formal validation techniques, receiving positive feedback from domain experts. Looking ahead, the authors plan to explore automated techniques for natural language processing to enhance the methodology's automation and scalability, ensuring its broader applicability in safety-critical domains.

The application of the methodology to the ETCS project demonstrated its effectiveness in improving requirement quality and increasing confidence in their correctness. The tool developed was validated by domain experts and received positive feedback for its potential applicability in larger contexts. The authors also discuss future work, including the automation of the initial phase and scalability of verification tools.

The authors of P3 propose an architecture-centric approach to model-based engineering for safety-critical embedded systems. They emphasize the need for early fault discovery in the development life cycle to reduce costs and improve system reliability. The key technology discussed is the Architecture Analysis & Design Language (AADL), an SAE International standard¹. AADL supports analysis of operational qualities like safety-criticality and reliability

through model annotations, enabling validation of systems from an architecture model.

The dataset mentioned includes system-level faults data, particularly those discovered late in the development life cycle. This data is used to analyze the root causes of system-level failures and validate the effectiveness of the AADL technology. Industry initiatives demonstrate the feasibility of using AADL in practice. The approach has shown promise in the Aerospace industry, with potential cost savings by identifying faults earlier in the life cycle and ensuring system safety and reliability.

Article P4 discusses the challenges in validating safety-critical embedded systems, particularly the late discovery of system-level faults and inconsistencies in model-based analysis results. It proposes an architecture-centric model-based analysis approach using the SAE Architecture Analysis & Design Language (AADL) standard. This approach aims to address root causes of system-level problems by analyzing different operational quality aspects from an architecture model.

The dataset mentioned is an industrial case study demonstrating the feasibility of the proposed approach. The case study, conducted by the Aerospace Vehicle Systems Institute (AVSI), involved creating and analyzing a multi-tier aircraft model. The analysis focused on system properties like mass, electrical power consumption, and computer resource allocations, as well as end-to-end latency analysis across subsystems.

The results showed that early and continuous virtual model integration based on standardized representations can reduce errors, ensure consistency, and leverage industrial investment in tools through well-defined interchange formats. The case study concluded that the SAVI concept is sound and should be implemented with further development, indicating a positive return on investment for adopting the architecture-centric paradigm.

P5 introduces SpecML, a novel modeling language aimed at enhancing the requirements specification process for avionics software. SpecML is designed to address three main objectives: enforcing certification information mandated by DO-178C, capturing requirements in natural language to facilitate industry adoption, and providing structured, semantically-rich formalism for requirements-based analyses and testing. To achieve this comprehensive solution, SpecML combines features from SysML, MARTE, and the property-based requirement (PBR) theory. By leveraging these approaches, SpecML aims to streamline the requirements specification process while ensuring compliance with industry standards.

The methodology employed involves the development of a domain metamodel and its integration with the UML profile. This iterative process ensures that the resulting SpecML model accurately represents the requirements of avionics software systems. To validate SpecML, the authors conducted an empirical evaluation using a case study of an aircraft's flight control software (FCS). In this evaluation, SpecML was used to model all requirements of the FCS and verify their compliance with DO-178C objectives. The results demonstrate the effectiveness of SpecML in facilitating requirement specifications and enabling requirements-based analyses and testing.

The authors of P6 discuss the joint use of static and dynamic software verification techniques across various safety-critical system industries. They explore how formal methods (FM) and testing are combined in the safety standards of the automotive, aeronautic, nuclear, process, railway, and space industries. The paper is a cross-domain analysis by experts involved in the revision committees of standards like ISO 26262, DO-178C, IEC 60880, IEC 61508, EN 50128, and ECSS-Q-ST-80C.

The approach suggested involves reviewing commonalities and differences in the application of formal methods within these standards, examining whether they are mandatory or recommended, the types of properties they should be applied to, and the coverage required if testing and formal methods are used together. The authors provide insights based on their experience in the industry regarding the state of the art practice of combining formal methods and testing, discussing where formal methods have outperformed testing, the role of structural coverage, and verification termination criteria in cases of FM-test mix.

The dataset used for this analysis comprises the safety standards themselves and the professional experiences of the group members. The results indicate that while the standards provide a range of guidelines on testing and formal methods,

the actual practice varies significantly, with some industries like railway and aeronautics effectively using formal methods in production software development. The paper concludes with prospective views on how software safety standards may evolve to maximize the benefits of dynamic (testing) and static (FM) verification methods. The authors suggest alternative means for unintended function detection, such as FM-verification of the specification, double independent specification, and enhanced exploratory testing, to improve property coverage and assumption coverage.

P7 delves into the crucial realm of validation and verification (V&V) techniques essential for ensuring the integrity and reliability of safety-critical software systems. The authors advocate for a comprehensive approach to V&V, emphasizing its necessity throughout the Software Development Life Cycle (SDLC) to ascertain both the correctness of the product and its adherence to specified requirements. To substantiate their assertions, the authors undertake a detailed examination of various V&V methodologies, aiming to identify the most effective techniques suitable for safety-critical software systems.

In their investigation, the authors explore a diverse range of V&V methodologies, encompassing effective software testing strategies, model-based approaches, and conventional-based techniques. Noteworthy methodologies discussed include Miller's formula for determining failure probability in scenarios where testing yields no defects, as well as the utilization of fault trees and event trees for testing safety-critical software systems. Additionally, the study presents an alternative verification procedure based on symbolic models, enabling comprehensive exploration of system state space and verification of property satisfaction. The authors further conduct a survey involving 50 participants from multiple companies, utilizing a questionnaire structured according to IEEE Standard 1012TM-2012, to gather insights into the efficacy of various V&V techniques.

The survey results serve as a cornerstone for evaluating and comparing V&V techniques such as Failure Mode and Effects Analysis (FMEA), Hazard and Operability Study (HAZOP), Fault Tree Analysis (FTA), and others. Through a meticulous analysis of participant responses, the study sheds light on the effectiveness and applicability of different V&V methodologies, with a particular focus on techniques like HAZOP that exhibit well-defined and widely applicable procedures. The findings underscore the critical role of V&V in ensuring the development of safe and reliable software systems, highlighting the importance of meticulous methodology in safeguarding against potential risks and ensuring compliance with safety standards.

Article P8 presents a method for designing safety-critical systems based on Boolean algebra, which are compliant with safety standards like IEC 61508 and EN 50128. The authors propose an optimization-based module design process that utilizes various optimization techniques and formal verification to ensure the system meets all defined safety rules while optimizing for performance and availability.

The method employs diverse optimization algorithms, including Estimation of Distribution Algorithms (EDA), Iterated Local Search (ILS), Ant Colony Optimization (ACO), and a hybrid approach, to find an optimal design that adheres to safety rules and performance criteria. The optimization process involves defining the problem, selecting an appropriate algorithm, solving the problem, and transforming the knowledge into a verifiable format.

A simplified railway interlocking case study is used to evaluate the method. The optimization problem is modeled as a binary optimization problem with objective functions related to safety, availability, and performance. The algorithms are tuned and applied to the case study, with the hybrid EDA and ILS method performing best. The formal verification activity confirms the safety of the design, allowing the use of non-qualified optimization tools in the design phase. The results demonstrate the effectiveness of the proposed method in facilitating the design of safety-critical systems that comply with safety standards, potentially reducing development and certification costs while mitigating human error. The approach's cross-domain applicability is highlighted, and the authors provide access to the case study software and results for further analysis and adaptation to other applications and domains.

P9 introduces the ASSURE framework, which aids in the formal verification of safety-critical aerospace systems. ASSURE provides rigorous verification of deterministic and non-deterministic properties for both centralized and decentralized aerospace applications, ensuring their safe operation in populated areas.

ASSURE employs formal theorem proving tools to verify the correctness of aerospace systems. It utilizes a variety of techniques, including Hoare logic, temporal logic, and first-order logic theorem proving, to model and reason about the systems' properties. The framework also includes a data-driven runtime verification approach to adapt to dynamic operating conditions.

While the article does not specify a particular dataset, it discusses the use of data-driven models for applications like stall detection in fixed-wing aircraft and rotor fault detection in hexacopters. The results include the development of formal proof libraries and the verification of algorithms and software that contribute to a provably safe Internet of Planes infrastructure.

The article P10 presents XANDAR, a comprehensive verification and validation (V&V) approach for safety-critical systems. The authors propose a strategy that integrates various tools and techniques to ensure system safety and security, particularly in the automotive and avionics industries. The XANDAR V&V strategy emphasizes early integration of simulation and static analysis techniques during the development cycle. This differs from traditional methods by focusing on early verification rather than later stages. The approach includes Static Analysis, Timing Analysis, Model-in-loop, and Network simulation tools to address complex system design challenges.

The authors utilize tools like Polyspace Bug Finder and Code Prover for static analysis, Timing Architect for timing analysis, and NS3 simulator for network architecture simulation. These tools help in ensuring functional correctness and meeting timing requirements, which are crucial for the reliability of safety-critical systems.

The verification approaches are validated by applying them to automotive and avionics use cases. The proposed XANDAR approach aims to reduce bug density and prove functional correctness early in the development process, thereby enhancing the safety and reliability of the system software. The article underscores the importance of rigorous V&V processes in the software development lifecycle to mitigate risks associated with safety-critical systems. The XANDAR project represents a significant effort to address the challenges posed by the increasing complexity and connectivity of such systems.

4. Results

Revisiting the research questions:

RQ1: What are the prevalent methods and tools used for validation and verification in safety-critical software systems?

The reviewed studies demonstrate a range of methodologies, such as model-based verification, static analysis, and formal methods. Tools such as Polyspace, as well as methodologies like FMEA (Failure Modes and Effects Analysis) and HAZOP (Hazard and Operability Study), are commonly employed in various applications.

RQ2: How effective are these methods in identifying and mitigating potential failures in such systems?

The effectiveness of these methods often varies significantly, with the specific application domain and system complexity playing a crucial role. The robustness of model-based verification and static analysis is particularly emphasized in their ability to detect potential failures at an early stage.

RQ3: What are the gaps in current research on software validation and verification for safety-critical systems?

The present literature review aims to identify significant deficiencies in the integration of validation and verification techniques across various domains, including aerospace and automotive industries. The effectiveness of techniques in one area often lacks validation in others, underscoring the necessity for broader applicability studies. In addition, there is a lack of research regarding the adaptation of these methods for real-world applications, which involve complex systems that are continuously updated. Furthermore, the increasing incorporation of artificial intelligence (AI) in safety-critical systems presents difficulties in managing the unpredictable behaviors of machine learning. This is a gap that existing methodologies must adapt to address. It is crucial to rectify these shortcomings in order to improve the dependability and suitability of these techniques in dynamic operational settings.

CRedit authorship contribution statement

Turcu Ciprian-Stelian: Conceptualization, Methodology, Validation, Resources, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization. **Turcu Remus-Gabriel:** Conceptualization, Methodology, Validation, Resources, Data Curation, Writing - Review & Editing, Visualization. **Țăpoi Marius-Ștefan:** Conceptualization, Validation, Investigation, Resources, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization, Project administration.

References

- [AQH15] Waqas Ahmad, Usman Qamar, and Shoaib Hassan. Analyzing different validation and verification techniques for safety critical software systems. In *2015 6th IEEE International Conference on Software Engineering and Service Science (ICSESS)*, pages 367–370. IEEE, 2015.
- [CRST10] Alessandro Cimatti, Marco Roveri, Angelo Susi, and Stefano Tonetta. Formalization and validation of safety-critical requirements. *arXiv preprint arXiv:1003.1741*, 2010.
- [Fei09] Peter H Feiler. Challenges in validating safety-critical embedded systems. *SAE International Journal of Aerospace*, 3(2009-01-3284):109–116, 2009.
- [Fei10] Peter H Feiler. Model-based validation of safety-critical embedded systems. In *2010 IEEE Aerospace Conference*, pages 1–10. IEEE, 2010.
- [KC07] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. 2, 01 2007.
- [KKW13] Arsalan H Khan, Zeashan H Khan, and Zhang Weiguo. Model-based verification and validation of safety-critical embedded real-time systems: Formation and tools. In *Embedded and Real Time System Development: A Software Engineering Perspective: Concepts, Methods and Principles*, pages 153–183. Springer, 2013.
- [LBA⁺14] Emmanuel Ledinot, Jean-Paul Blanquart, Jean-Marc Astruc, Philippe Baufreton, Jean-Louis Boulanger, Cyrille Comar, Hervé Delseny, Jean Gassino, Michel Leeman, Philippe Quéré, et al. Joint use of static and dynamic software verification techniques: a cross-domain view in safety critical system industries. In *Embedded Real Time Software and Systems (ERTS2014)*, 2014.
- [PCD⁺23] Saswata Paul, Elkin Cruz, Airin Dutta, Ankita Bhaumik, Erik Blasch, Gul Agha, Stacy Patterson, Fotis Kopsaftopoulos, and Carlos Varela. Formal verification of safety-critical aerospace systems. *IEEE Aerospace and Electronic Systems Magazine*, 2023.
- [PEB19] Andrés Paz and Ghizlane El Boussaidi. A requirements modelling language to facilitate avionics software verification and certification. In *2019 IEEE/ACM 6th International Workshop on Requirements Engineering and Testing (RET)*, pages 1–8. IEEE, 2019.
- [PFB⁺21] Jon Perez, Jose Luis Flores, Christian Blum, Jesús Cerquides, and Alex Abuin. Optimization techniques and formal verification for the software design of boolean algebra based safety-critical systems. *IEEE Transactions on Industrial Informatics*, 18(1):620–630, 2021.
- [SSS⁺23] Balmukund Sonigara, Sakir Sezer, Fahad Siddiqui, Raphael Weber, Konstantinos Antonopoulos, Christos Panagiotou, Christos P Antonopoulos, Georgios Keramidas, Nikolaos Voros, Sena Busra Yengoc-Tasdemir, et al. Xandar: Verification & validation approach for safety-critical systems. In *2023 IEEE 36th International System-on-Chip Conference (SOCC)*, pages 1–6. IEEE, 2023.

Id	Citation	Methods Used	Domains Applied	Key Findings	Tools
P1	[KKW13]	Simulation, Model-based Verification	Automotive, Aerospace, Avionics	Improved methods for detecting issues through simulation	Matlab/Simulink Embedded Target toolbox for Motorola PowerPC (ETMPC) Real-Time Workshop Embedded Coder (RTWEC) AeroSim + Aerospace Blocksets
P2	[CRST10]	Formal Methods, Requirement Validation	Railways	Demonstrated effectiveness of formal methods in validating critical requirements	RequisitePro Rational Software Architect (RSA) NuSMV/CEGAR EuRailCheck Tool (developed within EuRailCheck)
P3	[Fei09]	Empirical Studies, Challenges Analysis	Aerospace, Avionics, Aeronautic	Highlighted major challenges in validating safety-critical embedded systems	SAE AADL VHDL Simulink
P4	[Fei10]	Model-Based Engineering, Case Studies	Aerospace, Avionics	Provided insights on model-based validation effectiveness	SAE AADL OSATE AVSI SAVI
P5	[PEB19]	Requirements Modelling, HAZOP	Avionics	Discussed the benefits of using a requirements modelling language in avionics Validation & Verification	Eclipse Papyrus SysML MARTE MathWorks SpecML (developed)
P6	[LBA ⁺ 14]	Static and Dynamic Verification, Comparative Analysis	Aeronautic, Automotive, Nuclear, Process, Railway, Space	Offered insights into joint use of static and dynamic methods across industries	
P7	[AQH15]	Survey, Comparative Study	General Safety-critical Software Systems	Analyzed and compared different Validation & Verification techniques for safety-critical systems	Web-based questionnaire
P8	[PFB ⁺ 21]	Optimization Techniques, Formal Verification	Boolean Algebra based Systems	Explored optimization and verification techniques in boolean algebra systems	Simulink Design Verifier Prover Python
P9	[PCD ⁺ 23]	Formal Verification, Software Design Analysis	Aerospace	Assessed the formal verification processes in aerospace safety-critical systems	PILOTStoDafny Agda Athena ASSURE Framework (developed)
P10	[SSS ⁺ 23]	Static and Dynamic Analysis, Network Simulation	Automotive, Avionics	Described the use of static, dynamic, and network simulation in Validation & Verification processes	Polyspace Bug Finder Code Prover Fucsim Tool Timing Architect NS3 simulator XANDAR Toolchain (developed)

Table 3

Summary of the selected articles with detailed breakdown of methodologies, domains, and key findings.